



MODULE :

Le Cloud Computing & DevOps

Pr. F. Benabbou
Master DSBD & IA
2025-2026
Faculté des Sciences Ben M'Sik Casablanca





TABLE OF CONTENTS

01 CLOUD COMPUTING

- Introduction générale
- La Virtualisation
- Les concepts de base du Cloud Computing
- Technologies émergentes du CC : Edge, Fog, ...
- Étude de cas et projet pratique

02 DevOps & Cloud

- La philosophie DeVops
- Version control systems (git)
- Intégration Continue CI
- Tests automatisés
- Déploiement Continu CD
- Infrastructure en tant que Code (IaC)
- Surveillance et Journalisation
- Étude de cas et projet pratique



01

Développement Continue



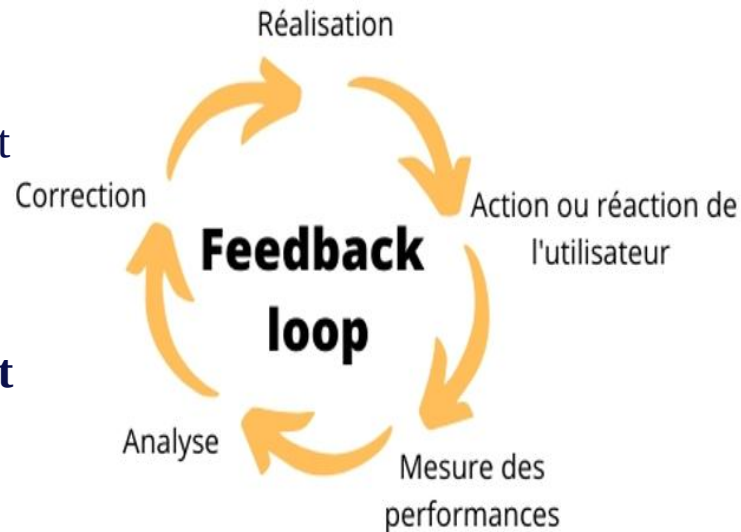
Développement Continu





Développement Continu

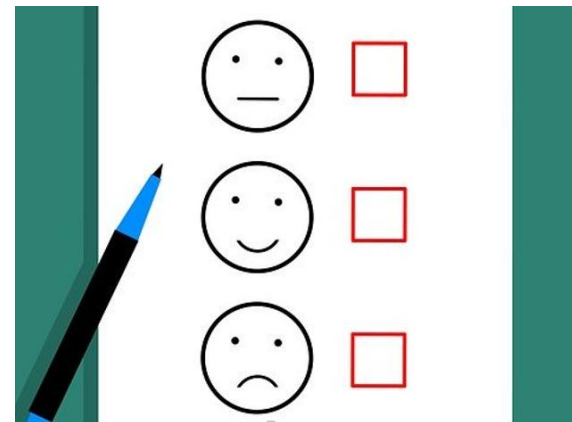
- Le développement continu est la première étape du cycle de vie DevOps et implique la **planification** et le **codage** des logiciels.
- le **processus** de développement complet est décomposé en **cycles** de développement plus petits
- Les logiciels ou les applications sont **fréquemment améliorés**, avec une plus grande **rapidité** et une meilleure réactivité aux **feedbacks** des utilisateurs.
- Les logiciels sont constamment améliorés, mis à jour et déployés de manière **itérative**.





Développement Continu

- Dans un environnement de développement continu, les **tests** sont automatisés et exécutés chaque fois qu'une **modification** est effectuée dans le code.
- Les tests peuvent être vus comme des "feedbacks techniques" sur la qualité du code développé.
- si un test échoue, il faut corriger les erreurs rapidement





Développement Continue et versionning

■ Risques & besoins

- Lorsque **plusieurs** développeurs travaillent sur la même code projet, les risques de **conflits** sont élevés, surtout s'ils travaillent sur les mêmes fichiers ou fonctions.
- si quelqu'un supprime ou **écrase** accidentellement des parties critiques du code, le travail peut être perdu et il n'y a pas de moyen de le récupérer.
- Durant l'avancement du projet, les développeurs peuvent avoir besoin de maintenir et de travailler sur **différentes versions** de l'application qu'il faut maintenir
- Il est souvent difficile de revenir à une dernière **version stable** en cas de problème.
- Manque de **visibilité** sur les modifications



Développement Continue et versionning

- Pour pouvoir **intégrer** le code en permanence, il faut d'abord pouvoir gérer les différentes **versions** du code, surtout lorsque plusieurs développeurs travaillent en même temps.
- C'est pour cela qu'on utilise un système de **gestion de versions**, qui permet :
 - ✓ d'enregistrer l'historique des modifications, de revenir en arrière si nécessaire, et de collaborer efficacement.
 - ✓ D'avoir une copie partagée de tous les fichiers de code à laquelle tous les utilisateurs peuvent accéder .
 - ✓ De voir quels fichiers ont été modifiés par d'autres, quand et de visualiser ces modifications et connaître les raisons.
 - ✓ De gérer les conflits lorsque plusieurs utilisateurs modifient le même fichier.
 - ✓ Etc.



Développement Continue et versionning

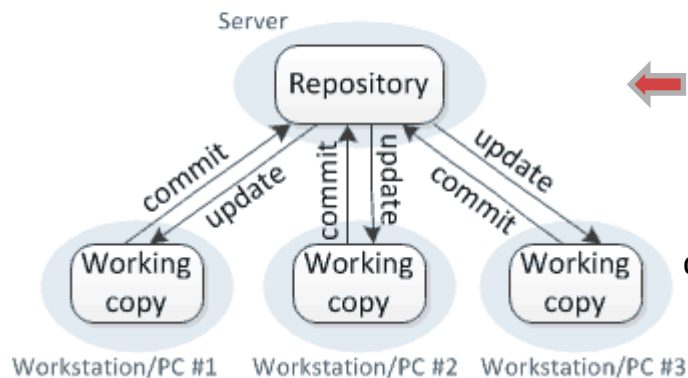
- De nombreux systèmes de contrôle des versions (VCS) sont conçus et utilisés spécialement pour les projets de génie logiciel
- Deux types : VCS Centralisé, VCS distribué
 - ✓ VCS Centralisé : un seul serveur central qui contient l'intégralité de l'historique des versions.
 - ✓ VCS distribué : Chaque développeur possède une copie complète du dépôt, avec tout l'historique sur sa machine, le dépôt est « cloné » et chaque ordinateur devient un miroir complet du serveur.

Contrôle de versions

■ Caractéristique

- ✓ Historique : Uniquement sur le serveur / Sur chaque poste de travail.
- ✓ Travail hors-ligne : Impossible / possible.
- ✓ Vitesse : Lent /Très rapide.
- ✓ Point de défaillance : le serveur central/ aucun

Centralized version control

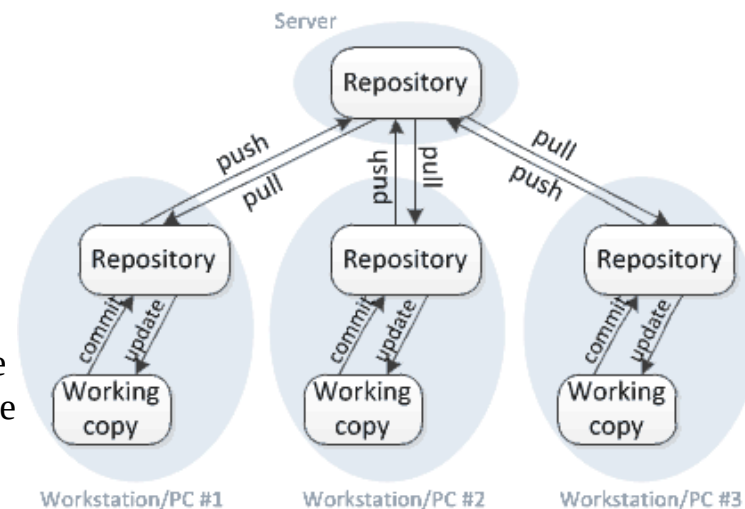


1. Aucun dépôt ne fait Autorité →
- ← 2. Un seule dépôt faisant autorité

Un **repository** est un espace de stockage où le code source d'un projet est conservé, versionné, et suivi.

Exemples: Concurrent Version System (CVS),
Subversion (SVN)

Distributed version control



Exemples: **Git**, Mercurial

Système de Contrôle de versions



- Git est un système de contrôle de version distribué et open source.
- Git a été inventé par Linus Torvalds, le créateur du noyau Linux, en 2005.
- L'histoire de Git est étroitement liée à l'évolution du développement du noyau Linux et aux problèmes rencontrés avec le système de gestion de version précédent (BitKeeper).



Système de Contrôle de versions



- Repository local :
 - C'est un dépôt Git sur la machine locale d'un développeur.
 - Il contient une copie complète du code source, ainsi que de son historique complet.
 - Un repository local permet aux développeurs de travailler de manière autonome et de synchroniser leurs modifications avec un repository distant lorsqu'ils le souhaitent.
- Repository distant :
 - Un repository Git stocké sur un serveur ou une plateforme comme GitHub, GitLab, ou Bitbucket.
 - Il permet aux développeurs de partager leur travail, de collaborer, et de gérer des projets à grande échelle.
 - Le repository distant sert souvent de source principale pour les équipes qui collaborent.



Concepts de base

■ Version

- Une version est un instantané du projet à un moment donné.
- Dans Git, chaque version est enregistrée sous la forme d'un **commit**.

■ Commit

- Un commit est appliqué lorsqu'on a terminé les modifications et on a une version stable du projet.
- Chaque commit contient un message décrivant les **changements** effectués (raison).

■ Fusion (Merge)

- consiste à **combiner** les **modifications** effectuées sur une branche dans une autre, pour intégrer les fonctionnalités développées dans des branches secondaires à la branche principale.

- **Branche** est une ligne indépendante de développement qui permet de travailler sur différentes fonctionnalités ou corrections sans affecter la version principale du projet.



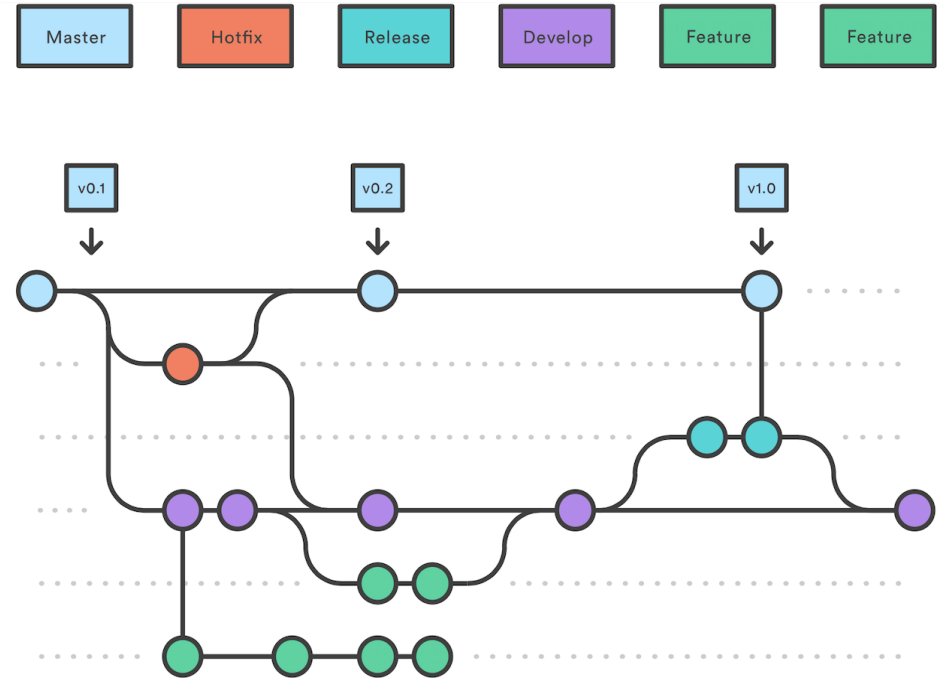
branche (Branch)

- Il y a deux types
 - Branche **principale** (**main** ou **master**)
 - ✓ elle doit être la branche la plus **stable**.
 - ✓ Chacune des modifications (effectuée avec **commit**) de cette branche est une nouvelle **version**
 - ✓ les développeurs ne travaillent jamais directement à partir de la branche master
 - Branches **secondaires** : personnalisables, elles sont utilisées pour des développements spécifiques
 - ✓ *Develop* est la branche d'intégration.
 - ✓ *Feature* est une branche pour créer une nouvelle fonctionnalité
 - ✓ *Release*, lorsque le contenu de develop est satisfaisant pour une nouvelle version du logiciel, on crée une branche release.
 - ✓ *Hotfix* est destinée à corriger rapidement des **bugs critiques** directement sur la branche **Master**.

Concepts de base: branche (Branch)



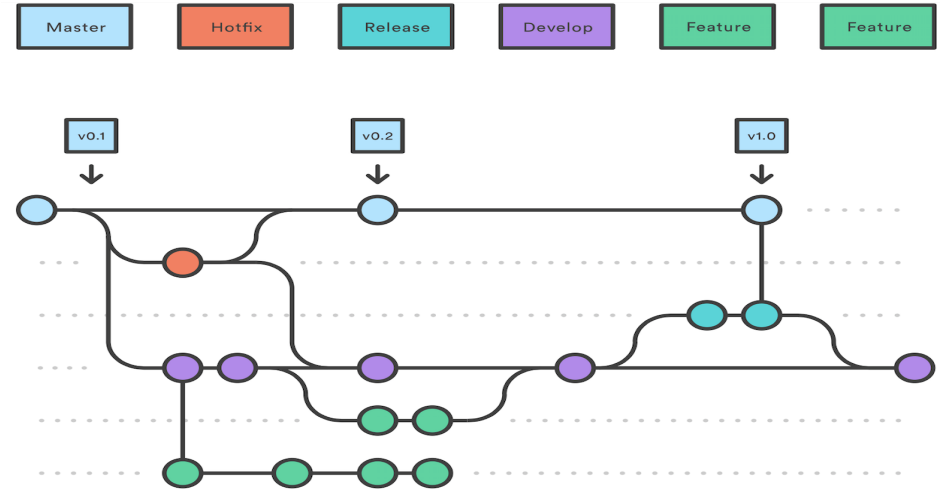
- Git ne propose pas directement des branches avec des nom spécifiques, les noms et les structures des branches sont totalement personnalisables.
- Afin de structurer le développement un modèle de workflow peut être utilisé appelé **Git Flow**.
- Git Flow est une convention de gestion des branches, qui automatise ce processus.



Concepts de base: branche (Branch)



- L'application est en version v0.1 qui est stable et publique.
- Dans **develop** on prépare la future version v1.0, on y rassemble toutes les nouvelles fonctionnalités terminées.
- Chaque développeur travaille sur une Fonctionnalité (feature), une fois le développement fini, la branche remonte vers **Develop**.
- Quand Develop contient assez de fonctions pour une mise à jour, on crée une **Release**.
- Une fois prête, elle est fusionnée dans **Master** et dans **Develop**



- Si un bug critique est découvert en production (ex: un bouton ne marche plus), on répare en urgence sur cette branche (Hotfix), on publie sur Master et on réinjecte la correction dans Develop.

Installer git



<http://git-scm.com/downloads>

The screenshot shows the Git Downloads page in a web browser. The browser's address bar displays `git-scm.com/downloads`. The page features the Git logo and the tagline "--fast-version-control". A search bar is located in the top right. On the left, a navigation menu includes links for "About", "Documentation", "Downloads" (which is highlighted), "GUI Clients", "Logos", and "Community". Below the menu, a text box mentions the "Pro Git book" by Scott Chacon and Ben Straub, available online for free. The main content area is titled "Downloads" and includes buttons for downloading on macOS, Windows, and Linux/Unix. A monitor graphic displays the "Latest source Release 2.47.1" with a "Download for Windows" button. Below this, a note states that older releases are available on GitHub. Further down, sections for "GUI Clients" and "Logos" are visible, each with a "View" link. At the bottom, the "Git via Git" section provides instructions on how to clone the repository using the command `git clone https://github.com/git/git` and mentions the web interface.



Comment ca marche?

- Créer un nouveau repo local

- Add et Commit

- Cycle de vie

- Branche/fusion

- Inspecter/Log

- Cloner un Repo

\$ git init

```
MINGW64:/d/git/ProjetDSBD
PC@DESKTOP-J1KT84B MINGW64 /d/git
$ cd ProjetDSBD/

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD
$ ls
LICENSE  bulletin-board-app/

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD
$ git init
Initialized empty Git repository in D:/Git/ProjetDSBD/.git/

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$
```





- Créer un nouveau repo local

- Add et Commit

- Cycle de vie

- Branche/fusion

- Inspector/Log

- Cloner un Repo

- Un fichier caché est crée .git qui contient tous les fichiers nécessaires au dépôt

```
MINGW64: d/git/ProjetDSBD/.git
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ cd .git

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/.git (GIT_DIR!)
$ ls
COMMIT_EDITMSG  config          hooks/  info/  objects/
HEAD            description    index  logs/  refs/

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/.git (GIT_DIR!)
$ ls -l
total 9
-rw-r--r-- 1 PC 197121 24 Jan 1 14:33 COMMIT_EDITMSG
-rw-r--r-- 1 PC 197121 23 Jan 1 13:40 HEAD
-rw-r--r-- 1 PC 197121 130 Jan 1 13:40 config
-rw-r--r-- 1 PC 197121 73 Jan 1 13:40 description
drwxr-xr-x 1 PC 197121 0 Jan 1 13:40 hooks/
-rw-r--r-- 1 PC 197121 225 Jan 1 14:33 index
drwxr-xr-x 1 PC 197121 0 Jan 1 13:40 info/
drwxr-xr-x 1 PC 197121 0 Jan 1 14:33 logs/
drwxr-xr-x 1 PC 197121 0 Jan 1 14:33 objects/
drwxr-xr-x 1 PC 197121 0 Jan 1 13:40 refs/

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/.git (GIT_DIR!)
$
```





- Créer un nouveau repo local

- Add et Commit

- Cycle de vie

- Branche/fusion

- Inspector/Log

- Cloner un Repo

\$ git add source
\$ git commit

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git add webapp.html styles.css

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git commit -m 'initial project version'
[master (root-commit) 57de69b] initial project version
2 files changed, 15 insertions(+)
create mode 100644 styles.css
create mode 100644 webapp.html

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$
```





- Créer un nouveau repo local

- Add et Commit

- Cycle de vie

- Branche/fusion

- Inspector/Log

- Cloner un Repo

- Ajouter un fichier quelconque dans le répertoire
- La command « status » permet de voir l'état des fichiers suivis et non suivis.

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  test.html
nothing added to commit but untracked files present (use "git add" to track)
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
```





Créer un nouveau
repo local

Add et Commit

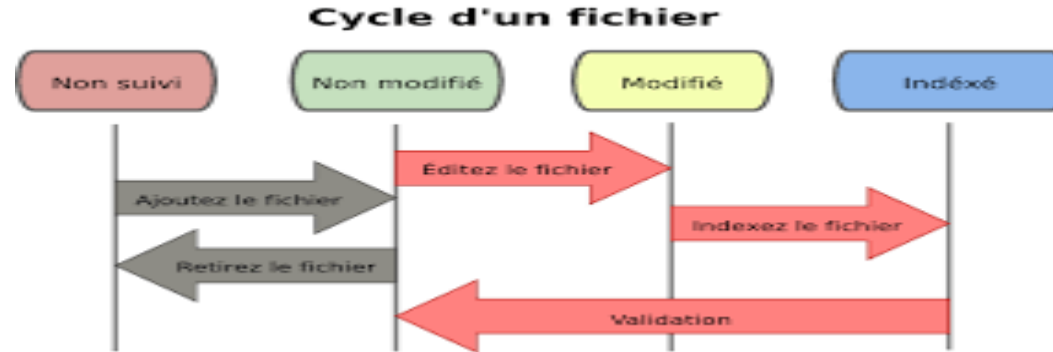
Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo

- **Untracked** (Non suivi) : Le fichier existe dans le répertoire de travail mais n'est pas encore suivi par Git.
- **Staged** (Indexé) : Le fichier a été ajouté à l'index avec la commande git add mais pas encore inclus dans un commit.
- **Non modifié ou Committed**: Le fichier a été enregistré dans l'historique du dépôt (dépôt local). Il est maintenant versionné et sa version actuelle est suivie par Git.
- **Modified** : Fichier est modifié mais non indexé.





- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspecter/Log
- Cloner un Repo

■ Exemple avec le fichier test.html.

```
MINGW64:/d/git/ProjetDSBD
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ ls -l
total 3
-rw-r--r-- 1 PC 197121 83 Jan  1 14:24 styles.css
-rw-r--r-- 1 PC 197121 170 Jan  1 15:55 test.html
-rw-r--r-- 1 PC 197121 182 Jan  1 14:25 webapp.html

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    test.html

nothing added to commit but untracked files present (use "git add" to track)

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git add test.html

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   test.html

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$
```





- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspector/Log
- Cloner un Repo

- Annuler l'indexation du fichier test.html.

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git reset

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    test.html

nothing added to commit but untracked files present (use "git add" to track)

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$
```



- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspecter/Log
- Cloner un Repo

- `git reset --soft HEAD~1` : annule le dernier commit mais conserve les changements dans l'index (comme s'ils étaient toujours staged).
- `git reset --hard` : Toutes les modifications non sauvegardées sont perdues.

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git reset

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.html

nothing added to commit but untracked files present (use "git add" to track)

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$
```





- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspecter/Log
- Cloner un Repo

■ Créer une nouvelle branche : `$git branch nom`

```
MINGW64:/d/git/ProjetDSBD

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git branch
* master

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git branch fonc1

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git branch
fonc1
* master

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$
```

L'astérisque indique la branche dans laquelle vous vous trouvez actuellement





- Pour aller sur la branche fonc1on utilise : `$ git switch fonc1`

```
MINGW64:/d/git/ProjetDSBD

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git switch fonc1
Switched to branch 'fonc1'

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (fonc1)
$ ls
styles.css  test.html  webapp.html

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (fonc1)
$
```

Créer un nouveau
repo local

Add et Commit

Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo



- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspecter/Log
- Cloner un Repo

■ Nous allons éditer et modifier le fichier webapp

```
MINGW64; d/git/ProjetDSBD
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (fonc1)
$ git status
On branch fonc1
changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   webapp.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.html

no changes added to commit (use "git add" and/or "git commit -a")
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (fonc1)
$
```

-a permet de ne pas utiliser le git add

■ Nous allons faire un commit

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (fonc1)
$ git commit -a -m "version0.1 avec un tableau"
[fonc1 52ddfff] version0.1 avec un tableau
1 file changed, 7 insertions(+), 1 deletion(-)

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (fonc1)
$
```



- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspecter/Log
- Cloner un Repo

- Nous avons deux branches ont divergentes, avec des modifications différentes sur le même fichier.
- On va essayer de fusionner les changements effectués dans `fonc1` avec `master`.

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git merge fonc1
Auto-merging webapp.html
CONFLICT (content): Merge conflict in webapp.html
Automatic merge failed; fix conflicts and then commit the result.

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   webapp.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .webapp.html.swp
    test.html

no changes added to commit (use "git add" and/or "git commit -a")
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master|MERGING)
$
```





Créer un nouveau
repo local

Add et Commit

Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo

- À ce niveau il faut résoudre le conflit et faire un add, commit





- Créer un nouveau repo local

- Add et Commit

- Cycle de vie

- Branche/fusion

- Inspecter/Log

- Cloner un Repo

- Revenir à la branche principale et modifier le fichier webapp

```
MINGW64:/d/git/ProjetDSBD

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master|MERGING)
$ git add webapp.html

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master|MERGING)
$ git commit -m "fusion de la version titre et tableau"
[master 98c7a30] fusion de la version titre et tableau
```





- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspecter/Log
- Cloner un Repo

- La Log est un outil important pour analyser l'historique
`$ git log --oneline --graph --decorate --all`

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git log --oneline --graph --decorate --all
* 98c7a30 (HEAD -> master) fusion de la version titre et tableau
| \
| * 52ddfff (fonc1) version0.1 avec un tableau
* | 159a015 changement du titre
| /
* 57de69b initial project version

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$
```

- Cela affiche les commits sous forme abrégée, avec une vue graphique, les noms des branches et de tous les commits.





- Créer un nouveau repo local
- Add et Commit
- Cycle de vie
- Branche/fusion
- Inspecter/Log
- Cloner un Repo

- La commande gitk est un outil graphique inclus avec Git, particulièrement utile pour avoir une vue d'ensemble visuelle des changements dans un dépôt :
 - L'historique des commits.
 - Les branches.
 - Les fusions.
 - Les modifications apportées à chaque commit.





ProjetDSBD: All files - gitk

File Edit View Help

master fusion de la version titre et tableau
fonc1 version0.1 avec un tableau
changement du titre
initial project version

Faouzia Benabbou <faouziat	2025-01-01 17:52:01
Faouzia Benabbou <faouziat	2025-01-01 17:26:33
Faouzia Benabbou <faouziat	2025-01-01 17:37:27
Faouzia Benabbou <faouziat	2025-01-01 14:33:13

SHA1 ID: 98c7a302bcd8fb6ba699d5ba609b81774ae33b0d ← → Row | 1 / 4

Find ↓ ↑ commit containing: Exact All fields

Search

☒ Diff ☐ Old version ☐ New version Lines of context: 3 ☐ Ignore space changes

Author: Faouzia Benabbou <faouziabenabbou@gmail.com> 2025-01-01 17:..
Committer: Faouzia Benabbou <faouziabenabbou@gmail.com> 2025-01-01..
Parent: 159a0154b03ecc925408e6d37666a6097342f1cd (changement du titr..
Parent: 52ddfff72d27bf5dd3dc262d47703ae15e832cc4 (version0.1 avec un..
Branch: master
Follows:
Precedes:

fusion de la version titre et tableau

webapp.html

index 8bcd6f7,9a79855..2a27459

@@@ -5,6 -5,12 +5,13 @@@

<link rel="stylesheet" type="text/css" href="styles.css">

</head>

<body>

- <h1> Master DSBD </h1>

++<h1> Master DSBD </h1>

+ <table>

Patch Tree

Comments

webapp.html

Créer un nouveau
repo local

Add et Commit

Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo

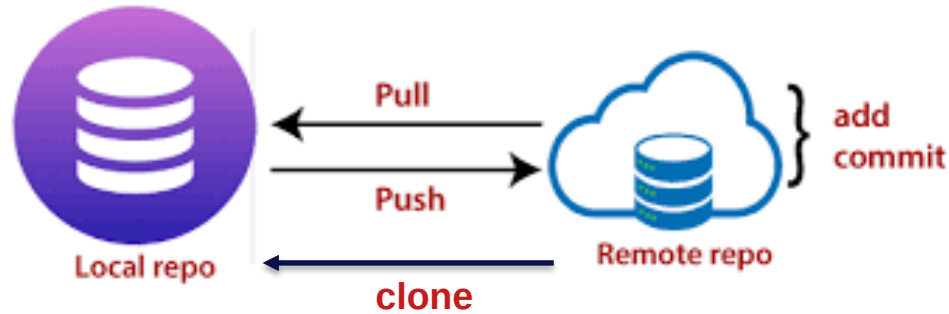




github



- Avec Github on peut partager ses codes et travailler en groupe



Créer un nouveau
repo local

Add et Commit

Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo



github



- Créer un nouveau repo local

- Add et Commit

- Cycle de vie

- Branche/fusion

- Inspector/Log

- Cloner un Repo

- Avec Github on peut partager ses codes et travailler en groupe
git clone <URL_du_dépôt>

```
MINGW64/d/git/ProjetDSBD
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ git clone https://github.com/faouziabenabbou/Syst-me-expert-Diagnos
tic.git
Cloning into 'syst-me-expert-Diagnostic'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 13 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0
)
Receiving objects: 100% (13/13), 16.75 KiB | 816.00 KiB/s, done.
Resolving deltas: 100% (3/3), done.
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD (master)
$ ls
Syst-me-expert-Diagnostic/  styles.css  test.html  webapp.html
```





GitHub : travailler en équipe



■ Modifier (mineur) le fichier mycin.pl

Créer un nouveau
repo local

Add et Commit

Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo

```
MINGW64/d/git/ProjetDSBD/Syst-me-expert-Diagnostic
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ vim mycin.pl

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   mycin.pl

no changes added to commit (use "git add" and/or "git commit -a")
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$
```

```
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ git commit -a -m "Amélioration de l'interaction"
[main 1e19620] Amélioration de l'interaction
1 file changed, 1 insertion(+), 1 deletion(-)

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$
```



GitHub : travailler en équipe



Créer un nouveau
repo local

Add et Commit

Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo

- La commande suivante permet de pousser les modification sur github
- **git push origin main**

```
MINGW64/d/git/ProjetDSBD/Syst-me-expert-Diagnostic
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ git config --global http.sslverify false

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ git push origin main
warning: ----- SECURITY WARNING -----
warning: | TLS certificate verification has been disabled! |
warning: -----
warning: HTTPS connections may not be secure. See https://aka.ms/gcm/tlsverify for more info
rmation.
info: please complete authentication in your browser...
warning: ----- SECURITY WARNING -----
warning: | TLS certificate verification has been disabled! |
warning: -----
warning: HTTPS connections may not be secure. See https://aka.ms/gcm/tlsverify for more info
rmation.
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 320 bytes | 320.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/faouziabenabbou/Syst-me-expert-Diagnostic.git
  979cc22..1e19620  main -> main

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$
```





GitHub : travailler en équipe



- On vérifie sur GitHub

 **Syst-me-expert-Diagnostic** Public Pin Unwatch 1

 **main**  1 Branch  0 Tags t Add file <> Code

 Faouzia Benabbou Amélioration de l'interaction 1e19620 · 21 minutes ago 5 Commits	
 LICENSE	Initial commit 7 months ago
 README.md	Create README.md 2 months ago
 mycin.pl	Amélioration de l'interaction 21 minutes ago

 **README**  GPL-3.0 license 





GitHub : travailler en équipe



- Les collaborateurs doivent executer un pull pour avoir la dernière version modifiée.

\$git pull origin main

```
MINGW64:/d/git/ProjetDSBD/Syst-me-expert-Diagnostic
PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ git config --global http.sslverify true

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ git push origin main
Everything up-to-date

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$ git pull origin main
From https://github.com/faouziabenabbou/Syst-me-expert-Diagnostic
* branch          main          -> FETCH_HEAD
Already up to date.

PC@DESKTOP-J1KT84B MINGW64 /d/git/ProjetDSBD/Syst-me-expert-Diagnostic (main)
$
```



Créer un nouveau
repo local

Add et Commit

Cycle de vie

Branche/fusion

Inspector/Log

Cloner un Repo



TABLE OF CONTENTS

01 CLOUD COMPUTING

- Introduction générale
- La Virtualisation
- Les concepts de base du Cloud Computing
- Technologies émergentes du CC : Edge, Fog, ...
- Étude de cas et projet pratique

02 DevOps & Cloud

- La philosophie DeVops
- Version control systems (git)
- **Intégration Continue CI**
- **Tests automatisés**
- **Déploiement Continu CD**
- Infrastructure en tant que Code (IaC)
- Surveillance et Journalisation
- Étude de cas et projet pratique